COMP1752 Report

Introduction

This coursework is about creating a simple jukebox program using Python. The program lets users view tracks, create their own playlist, and update the ratings of different songs using a graphical interface. The main aim of this project was to understand the basics of object-oriented programming and build a working program that follows the given requirements. While working on this, I developed different parts of the system step by step, including viewing tracks, adding songs to a playlist, and updating ratings. I also added some improvements, such as checking for invalid input, to make the program more user-friendly.

Design and Development

This section explains how the program was designed and developed. The program used object-oriented programming by organising the code into classes such as TrackViewer, CreateTrackList, and UpdateTracks, with each class handling a different part of the system. The starting point of the project was the given view_tracks.py file, which helped me understand how the graphical interface works using Tkinter. This file already allowed users to view all tracks and check details of a specific track, so it gave me a good idea of how to build the rest of the system.

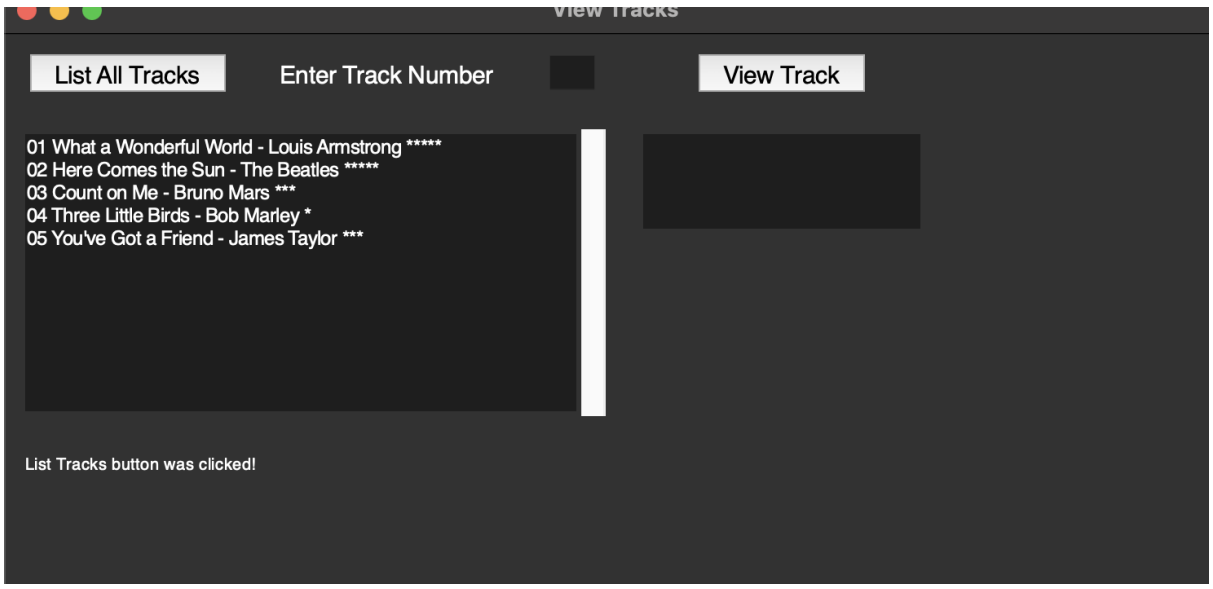


Figure 1

I then created two new parts of the program called create_track_list.py and update_tracks.py. the create_track_list.py file allows the user to enter a track number and add it to playlist.

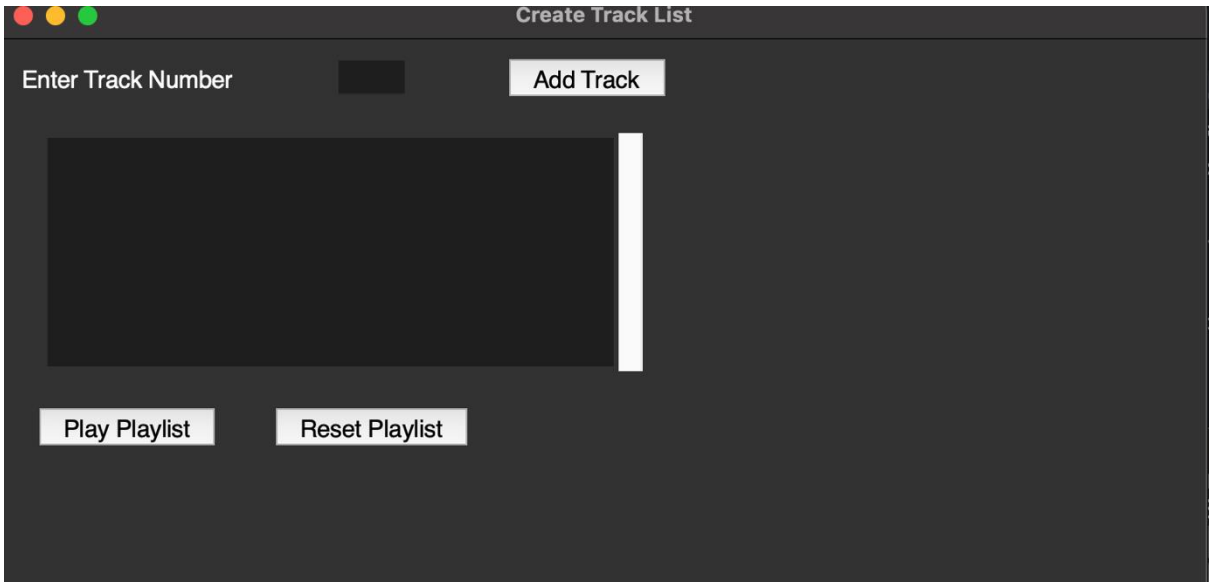


Figure 2

The playlist is shown in a text area so that the user can see all tracks that have been added.

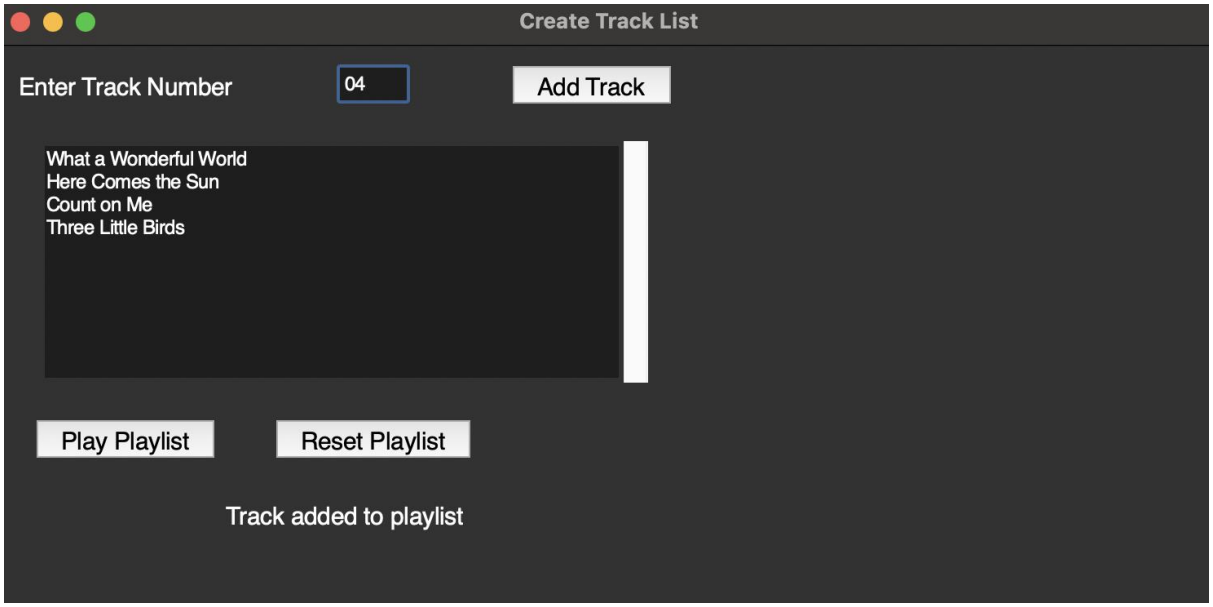


Figure 3

The user can also reset the playlist or simulate playing it.

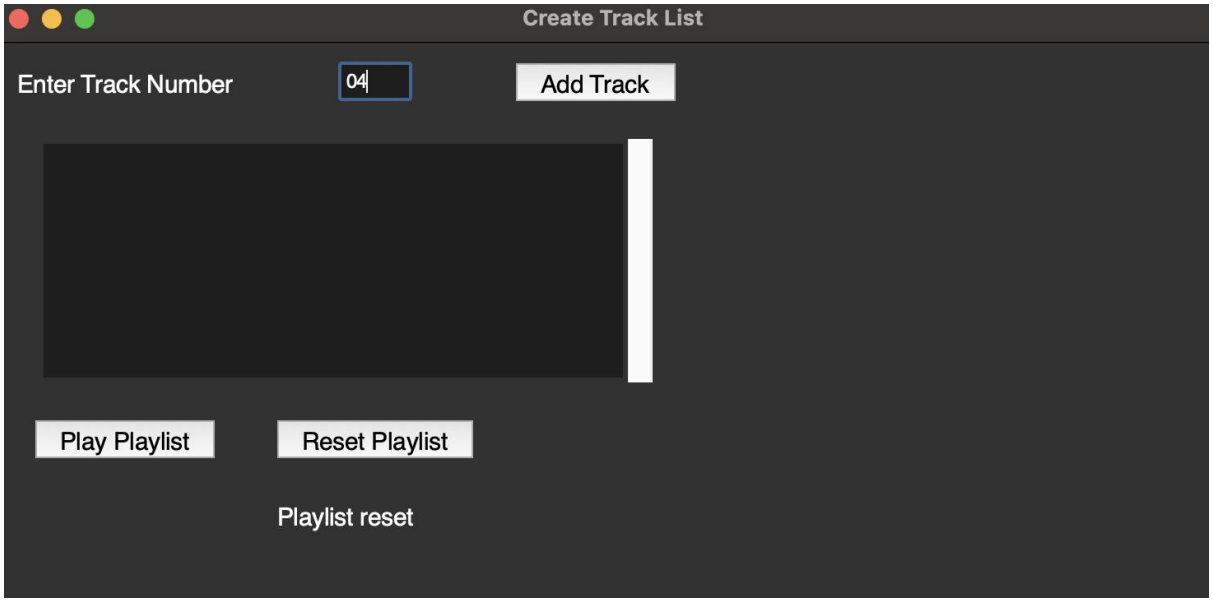


Figure 4

When the playlist is played, the play count for each track is increased. The update_tracks.py file allows the user to enter a track number and update its rating.

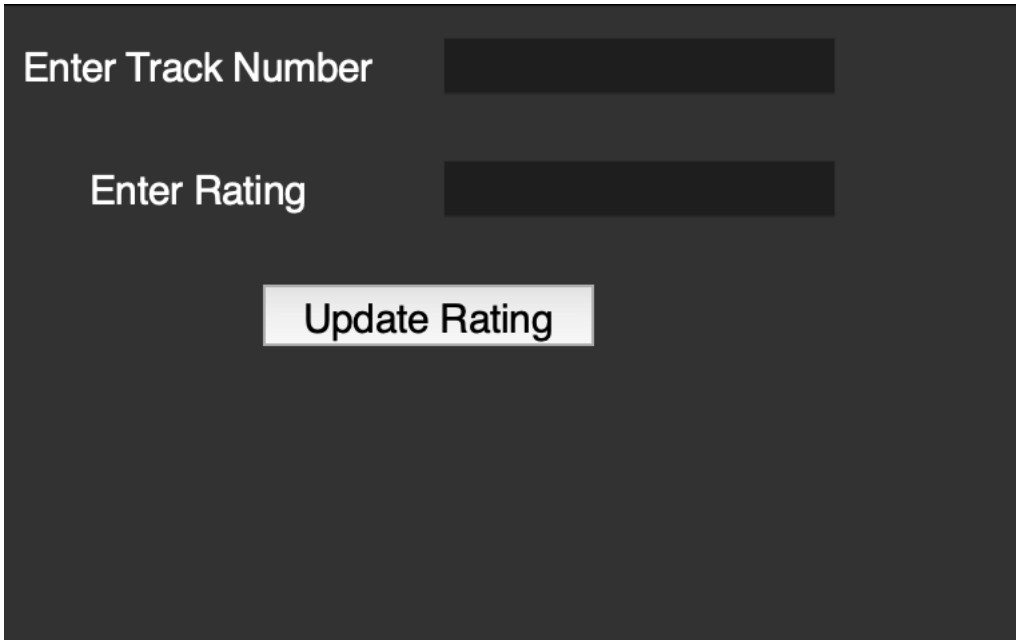


Figure 5

If the track exists, the new rating is shown along with the play count, and if it does not exist, an error message is displayed.

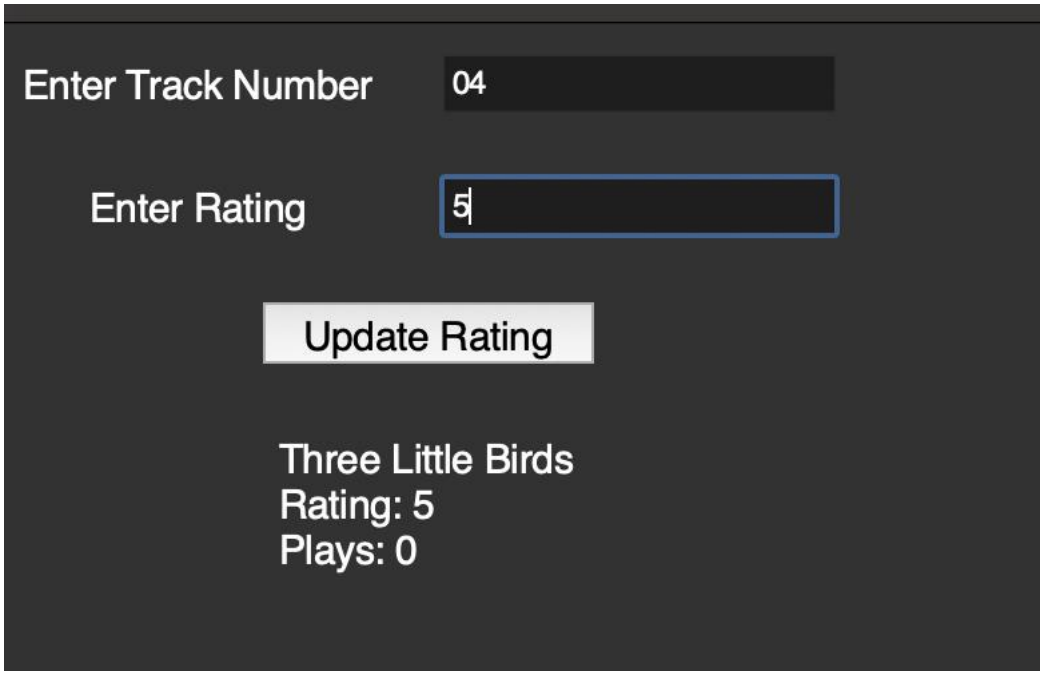


Figure 6

To improve the program, I added input validation to check that the user enters valid values. For example, if a user enters letters instead of numbers, the program shows an error message instead of crashing. This makes the program easier to use and more reliable.

Testing and Faults

This section explains how the program was tested to make sure it works correctly. Different inputs were used to check both normal and incorrect cases. For example, valid track numbers were entered to check if the correct track details were displayed, and invalid inputs such as non-existent track numbers or letters were used to check if error messages were shown. The program handled most cases correctly and displayed appropriate messages when errors occurred. Input validation was also added to prevent users from entering incorrect values, which improved the reliability of the system.

Appendix B:

View tracks

Input	Action	Expected Output	Actual Output
01	Click View Track	Track details shown	Works correctly
99	Click View Track	Error message	Works correctly

Create Track List

Input	Action	Expected Output	Actual Output
01	Click Add Track	Track added to playlist	Works correctly
99	Click Add Track	Error message	Works correctly
N/A	Click Reset Playlist	Playlist cleared	Works correctly
Playlist	Click Play Playlist	Play count increases	Works correctly

Update Tracks

Input	Action	Expected Output	Actual Output
01, 5	Click Update Rating	Rating Updated	Works correctly
abc, 5	Click Update Rating	Invalid input message	Works correctly

Conclusion & Reflection

This coursework helped me develop a better understanding of how to build a working program using Python and object-oriented programming. At the start, I found it quite difficult to understand how the code worked, especially when working with the graphical interface and connecting different parts of the program together. However, as I worked through each stage step by step, I became more confident in understanding how the program functions.

One of the more challenging parts was learning how to connect buttons to functions and making sure everything was in the correct place within the code. Small mistakes, such as incorrect syntax or placing code in the wrong section, caused errors, but fixing these helped me understand how the structure of the program works. On the other hand, once I understood the basic pattern of how GUI works, creating the different windows and features became much easier.

I also improved my problem-solving skills by testing the program and fixing errors when they appeared. Adding features such as input validation helped make the program more reliable and user-friendly. Overall, this coursework has helped me build confidence in coding, and I now have a better understanding of how to read and write Python programs. If I had more time, I would improve the program further by adding more features, such as searching for tracks or improving the design of the interface.